Stat 6341.001: Numerical Linear Algebra and Statistical Computing, Fall 2026 **Topic 1**

Numerical Linear Algebra and Statistical Computing

Lecturer: Yun Wei *Scribe: Students in Fall 2026*

Disclaimer: *These notes have not been subjected to the usual scrutiny reserved for formal publications. They may be distributed outside this class only with the permission of the Instructor.*

Contents

1 Multi-Armed Bandit **5**
- 1.1 Introduction . . . 5
  - 1.1.1 Multi-Armed Bandit Protocol . . . 5
  - 1.1.2 Simple Example: Always Pure Exploitation . . . 6
  - 1.1.3 Estimating f^* via Empirical Means . . . 6
  - 1.1.4 The Greedy Algorithm and Exploration . . . 6
  - 1.1.5 Failure of the Greedy Algorithm . . . 7
- 1.2 ϵ-Greedy Algorithm . . . 7
  - 1.2.1 Sub-Gaussian Concentration Bounds . . . 7
- 1.3 Upper Confidence Bound (UCB) Algorithm . . . 9

2 Unconstrained optimization **15**
- 2.1 Newton's method for unconstrained minimization (Section 9.5) . . . 15
  - 2.1.1 The Newton step is a descent direction . . . 15
  - 2.1.2 One-dimensional intuition . . . 15
  - 2.1.3 First interpretation: minimizing a quadratic approximation . . . 16
  - 2.1.4 Second interpretation: linearizing the optimality condition . . . 16
- 2.2 Newton decrement and stopping criterion . . . 16
  - 2.2.1 Newton's method algorithm . . . 17
- 2.3 Convergence discussion . . . 17

3 Equality constrained optimization **19**
- 3.1 Equality constrained minimization . . . 19
  - 3.1.1 Introduction . . . 19
  - 3.1.2 Equality constrained Convex Optimization . . . 20
  - 3.1.3 Equality constrained convex quadratic minimization . . . 21
- 3.2 Eliminating equality constraints . . . 24
  - 3.2.1 Review: equality-constrained minimization . . . 24
  - 3.2.2 Review: Quadratic objective . . . 24
  - 3.2.3 Parameterizing the feasible set (Section 10.1.2) . . . 24
  - 3.2.4 Optimal allocation with a resource constraint (Example 10.1) . . . 24

4 PCA **27**
- 4.1 Introduction . . . 27
- 4.2 Sample Covariance Matrix . . . 27
  - 4.2.1 Understand Σ_n (or C_n) . . . 28
- 4.3 PCA Algorithm . . . 28
- 4.4 In What Sense PCA Reorganizes the Data . . . 29
  - 4.4.1 Recall Affine Space . . . 29
  - 4.4.2 Least Square Metric . . . 30

4.4.3 PCA Minimizes the Least Square Metric 30
4.5 Complexity of the PCA Algorithm 31
4.6 Singular Value Decomposition (SVD) 31

Chapter 1

Multi-Armed Bandit

1.1 Introduction

The multi-armed bandit problem is a sequential decision-making formulation where an agent chooses between A distinct actions (arms) at each time step to maximize cumulative reward (or minimize cumulative regret):

- A available decisions/actions.
- Select an action at each round.
- Receive a stochastic, noisy reward.

1.1.1 Multi-Armed Bandit Protocol

For time steps $t = 1, 2, \ldots, T$:

1. Select a decision $\pi^t \in \Pi \coloneqq \{1, 2, \ldots, A\} = [A]$.
2. Observe a random reward $r^t \in \mathbb{R}$.

Here, t denotes the current time step, T is the total number of time steps, and A is the total number of available actions.

History The interaction history H_{t-1} contains all action-reward pairs observed up to time $t-1$:

$$H_{t-1} \coloneqq \{(a_1, r_1), (a_2, r_2), \ldots, (a_{t-1}, r_{t-1})\}$$

Assumption: Stochastic Reward The reward r_t is a random variable drawn from an underlying distribution conditioned on the interaction history:

$$r_t \sim M^*(\cdot \mid H_{t-1})$$

We briefly recall the multi-armed bandit setup and the ε-Greedy algorithm from the previous lecture. Let $f^*(\pi)$ denote the mean reward of action π, and let $\pi^* = \arg\max_{\pi \in \Pi} f^*(\pi)$ be an optimal action.

Reward Function The true expected reward function f^* for selecting action π is defined as:

$$f^*(\pi) \coloneqq \mathbb{E}[r \mid \pi]$$

Optimal Action The optimal decision π^* yields the highest expected reward:

$$\pi^* \coloneqq \arg\max_{\pi \in \Pi} f^*(\pi)$$

Regret The cumulative regret $\mathrm{Reg}(T)$ measures performance loss relative to the optimal action choice:

$$\mathrm{Reg} \coloneqq \sum_{t=1}^{T} f^*(\pi^*) - \sum_{t=1}^{T} \mathbb{E}_{\pi^t \sim p^t}[f^*(\pi^t)]$$

The first term is the total mean reward we could obtain by always choosing the optimal action π^*, whereas the second term is the expected total mean reward obtained by the algorithm. Thus, the goal is to minimize the regret.

Objective The goal is to design an algorithm achieving sub-linear regret, meaning the average regret vanishes asymptotically:

$$\lim_{T \to \infty} \frac{\mathbb{E}[\mathrm{Reg}(T)]}{T} = 0$$

1.1.2 Simple Example: Always Pure Exploitation

Suppose we always select the first decision, $\pi^t = 1$, for all $t = 1, 2, \ldots, T$. The total regret simplifies to:

$$\begin{aligned} \mathrm{Reg}(T) &\coloneqq \sum_{t=1}^{T} f^*(\pi^*) - \sum_{t=1}^{T} f^*(1) \\ &= T\big(f^*(\pi^*) - f^*(1)\big) = \mathcal{O}(T) \end{aligned}$$

This yields linear regret $\mathcal{O}(T)$, failing our sub-linear objective whenever action 1 is suboptimal.

1.1.3 Estimating f^* via Empirical Means

Since the true reward distribution f^* is unknown, we estimate it using the empirical average of past rewards:

$$\hat{f}^t(\pi) \coloneqq \frac{1}{n^t(\pi)} \sum_{s<t} r^s \mathbf{1}_{\{\pi^s = \pi\}}$$

where $n^t(\pi) \coloneqq \sum_{s<t} \mathbf{1}_{\{\pi^s = \pi\}}$ denotes the total number of times decision π was selected up to time t. The indicator function is defined as: $\mathbf{1}_{\{\pi^s = \pi\}} = \begin{cases} 1, & \pi^s = \pi, \\ 0, & \pi^s \neq \pi. \end{cases}$

1.1.4 The Greedy Algorithm and Exploration

A naive approach is the **Greedy Algorithm**, which chooses the decision maximizing the empirical reward estimate at step t:

$$\hat{\pi}^t = \arg\max_{\pi \in \Pi} \hat{f}^t(\pi)$$

While greedy selection exploits current knowledge, it lacks a mechanism for exploration. Because feedback is partial (rewards are observed only for chosen actions), the algorithm can prematurely abandon optimal actions due to early noisy estimates. Once an optimal action's empirical reward drops below another arm's estimate, it is never selected again, leading to linear regret $\mathcal{O}(T)$.

1.1.5 Failure of the Greedy Algorithm

Consider a two-armed bandit setting with $\Pi = \{1, 2\}$:

- Decision 1 yields a deterministic reward: $f^*(1) = \frac{1}{2}$.
- Decision 2 yields a stochastic reward: $r \sim \text{Ber}\left(\frac{3}{4}\right)$, so $f^*(2) = \frac{3}{4}$.

Consider the following trajectory:

1. At $t = 1$, choose Decision 1: receive $r^1 = \frac{1}{2}$. Thus, $\hat{f}^2(1) = \frac{1}{2}$.
2. At $t = 2$, choose Decision 2: receive $r^2 = 0$ (occurring with probability $\frac{1}{4}$). Thus, $\hat{f}^3(2) = 0$.
3. For all $t \geq 3$, the greedy strategy continuously chooses Decision 1 because $\hat{f}^t(1) = \frac{1}{2} > 0 = \hat{f}^t(2)$.

Under this event, the expected regret grows linearly:

$$\text{Reg}(T) = \left(\frac{3}{4} - \frac{1}{2}\right) + \left(\frac{3}{4} - \frac{3}{4}\right) + (T-2)\left(\frac{3}{4} - \frac{1}{2}\right) = \frac{T-1}{4} = \mathcal{O}(T)$$

1.2 ϵ-Greedy Algorithm

To prevent premature convergence to suboptimal choices, the ϵ-greedy algorithm forces exploration:

Exploitation With probability $1-\epsilon$, exploit current knowledge by picking the empirically optimal action:

$$\hat{\pi}^t = \arg\max_{\pi \in \Pi} \hat{f}^t(\pi)$$

Exploration With probability ϵ, explore uniformly at random across all available actions:

$$\hat{\pi}^t \sim \text{Unif}(\{1, 2, \ldots, A\})$$

Thus, with probability $1-\varepsilon$ the algorithm uses the currently best estimated action, while with probability ε it explores.

1.2.1 Sub-Gaussian Concentration Bounds

Definition 1.2.1 (σ-sub-Gaussian Variable)**.** A zero-mean random variable Z is σ-sub-Gaussian if its Moment Generating Function (MGF) satisfies:

$$\mathbb{E}\left[e^{\eta Z}\right] \leq \exp\left(\frac{\sigma^2 \eta^2}{2}\right), \quad \forall \eta \in \mathbb{R}$$

Lemma 1.2.2 (Sub-Gaussian Hoeffding Bound)**.** *Let $Z_1, \ldots, Z_T$ be independent, zero-mean σ-sub-Gaussian random variables. Then for any $u > 0$:*

$$\mathbb{P}\left(\frac{1}{T}\sum_{i=1}^{T} Z_i \geq u\right) \leq \exp\left(-\frac{Tu^2}{2\sigma^2}\right)$$

Proposition 1.2.3. *Assume that $f^*(\pi) \in [0,1]$ and the reward r^t is 1-sub-Gaussian (always think of it as Gaussian with variance 1). Then, for any finite T, by choosing ε appropriately, the ε-greedy algorithm ensures that with probability at least $1-\delta$,*

$$\mathrm{Reg} \lesssim A^{1/3}T^{2/3}\log^{1/3}\left(\frac{AT}{\delta}\right)$$

Consequently,

$$E\left[\mathrm{Reg}\right] \lesssim A^{1/3}T^{2/3}\log^{1/3}\left(AT\right)$$

To prove the proposition, let us consider the following lemma.

Lemma 1.2.4. *With probability at least $1-\delta$,*

$$\max_{\pi}\left|\widehat{f}^t(\pi) - f^*(\pi)\right| \lesssim \sqrt{\frac{A\log(AT/\delta)}{\varepsilon t}}$$

Comment: The left-hand side measures the largest estimation error over all actions. As t becomes larger, more observations are collected, so the estimation error becomes smaller. For a proof of the lemma, see Section 2.2 of [2].

Proof of Proposition 2.1

We begin with

$$\mathrm{Reg} = \sum_{t=1}^{T}\mathbb{E}_{\pi^t\sim p^t}\left[f^*(\pi^*) - f^*(\pi^t)\right].$$

Using the definition of the ε-greedy algorithm, this can be decomposed as

$$\mathrm{Reg} = \sum_{t=1}^{T}(1-\varepsilon)\left[f^*(\pi^*) - f^*(\widehat{\pi}^t)\right] + \varepsilon\sum_{t=1}^{T}\mathbb{E}_{\pi^t\sim\mathrm{unif}([A])}\left[f^*(\pi^*) - f^*(\pi^t)\right].$$

Since $f^*(\pi)\in[0,1]$,$f^*(\pi^*) - f^*(\pi^t) \le 1$. Therefore,

$$\mathrm{Reg} \le \sum_{t=1}^{T}\left[f^*(\pi^*) - f^*(\widehat{\pi}^t)\right] + \varepsilon T.$$

Now decompose

$$f^*(\pi^*) - f^*(\widehat{\pi}^t) = \left[f^*(\pi^*) - \widehat{f}^t(\pi^*)\right] + \left[\widehat{f}^t(\pi^*) - \widehat{f}^t(\widehat{\pi}^t)\right] + \left[\widehat{f}^t(\widehat{\pi}^t) - f^*(\widehat{\pi}^t)\right].$$

Because $\widehat{\pi}^t = \arg\max_\pi \widehat{f}^t(\pi)$, we have $\widehat{f}^t(\pi^*) - \widehat{f}^t(\widehat{\pi}^t) \le 0$. Hence,

$$f^*(\pi^*) - f^*(\widehat{\pi}^t) \le 2\max_{\pi}\left|f^*(\pi) - \widehat{f}^t(\pi)\right|.$$

Therefore,

$$\mathrm{Reg} \le 2\sum_{t=1}^{T}\max_{\pi}\left|f^*(\pi) - \widehat{f}^t(\pi)\right| + \varepsilon T.$$

Using the lemma 2.2, with probability at least $1-\delta$

$$\text{Reg} \lesssim 2\sum_{t=1}^{T}\sqrt{\frac{A\log(AT/\delta)}{\varepsilon t}} + \varepsilon T.$$

Using the fact $\sum_{t=1}^{T}\frac{1}{\sqrt{t}} \leq 2\sqrt{T}-1$, we obtain

$$\text{Reg} \lesssim 2\sqrt{\frac{A\log(AT/\delta)}{\varepsilon}}\sqrt{T} + \varepsilon T.$$

Comment: The first term is associated with the estimation error, while the second term comes from random exploration.

To balance these two terms, choose $\varepsilon \propto \left(\frac{A\log(AT/\delta)}{T}\right)^{1/3}$.

Plugging this choice into the previous bound gives

$$\text{Reg} \lesssim A^{1/3}T^{2/3}\log^{1/3}\left(\frac{AT}{\delta}\right).$$

This leads to the following question: can we improve the algorithm? The ε-greedy algorithm explores with probability ε, even for those well-estimated bad arms/actions. Can we choose actions in a more adaptive way instead of exploring uniformly?

1.3 Upper Confidence Bound (UCB) Algorithm

Suppose that at each time t we can construct a lower and upper confidence bound

$$\underline{f}^t, \overline{f}^t : \Pi \to \mathbb{R}$$

such that, with probability at least $1-\delta$,

$$f^*(\pi) \in \left[\underline{f}^t(\pi), \overline{f}^t(\pi)\right]$$

for every $t \in [T]$ and every $\pi \in [A]$.

The UCB algorithm chooses $\pi^t = \arg\max_{\pi\in\Pi}\overline{f}^t(\pi)$. Thus, at time t, the chosen action is the one having the largest upper confidence bound.

Lemma 1.3.1 (Instantaneous regret of UCB)**.** *For t, suppose $f^*(\pi) \in \left[\underline{f}^t(\pi), \overline{f}^t(\pi)\right]$*
Then $\pi^t = \arg\max_\pi \overline{f}^t(\pi)$ has the following property:

$$f^*(\pi^*) - f^*(\pi^t) \leq \overline{f}^t(\pi^t) - f^*(\pi^t) \leq \overline{f}^t(\pi^t) - \underline{f}^t(\pi^t).$$

Proof. Because π^t maximizes the upper confidence bound, $\overline{f}^t(\pi^*) \leq \overline{f}^t(\pi^t)$. Also, because the confidence interval contains the true mean reward, $f^*(\pi^*) \leq \overline{f}^t(\pi^*)$. Combining these gives $f^*(\pi^*) \leq \overline{f}^t(\pi^t)$. Therefore,

$$f^*(\pi^*) - f^*(\pi^t) \leq \overline{f}^t(\pi^t) - f^*(\pi^t).$$

Finally, $f^*(\pi^t) \geq \underline{f}^t(\pi^t)$, so $\overline{f}^t(\pi^t) - f^*(\pi^t) \leq \overline{f}^t(\pi^t) - \underline{f}^t(\pi^t)$.

□

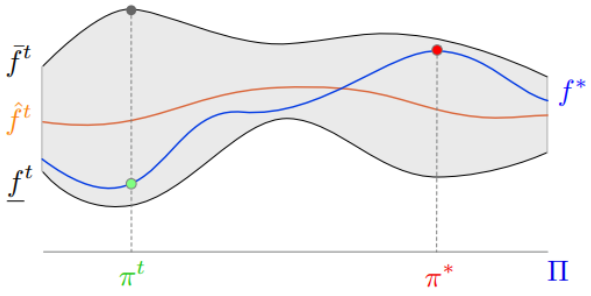


Figure 1.1: Illustration of the UCB algorithm.

Lemma 1.3.2 (Construction of confidence bounds)**.** *With probability 1-δ,*

$$\max \left| \widehat{f}^t(\pi) - f^*(\pi) \right| \leq \sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi)}}.$$

We skip the proof. Then by the lemma,

$$\underline{f}^t(\pi) := \widehat{f}^t(\pi) - \sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi)}} \leq f^*(\pi) \leq \widehat{f}^t(\pi) + \sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi)}} =: \overline{f}^t(\pi)$$

UCB:

Define

$$\widehat{f}^{\,t}(\pi) = \frac{1}{n^t(\pi)} \sum_{s<t} r^s \mathbf{1}(\pi^s = \pi)\,.$$

The upper confidence bound is

$$\overline{f}^{\,t}(\pi) = \widehat{f}^{\,t}(\pi) + \sqrt{\frac{2\log\left(\frac{2\pi^2 t^2 A}{\delta}\right)}{n^t(\pi)}}.$$

Similarly, the lower confidence bound is

$$\underline{f}^{\,t}(\pi) = \widehat{f}^{\,t}(\pi) - \sqrt{\frac{2\log\left(\frac{2\pi^2 t^2 A}{\delta}\right)}{n^t(\pi)}}.$$

With probability at least $1-\delta$,

$$\underline{f}^{\,t}(\pi) \leq f^\star(\pi) \leq \overline{f}^{\,t}(\pi).$$

The UCB action selection rule is

$$\pi^t = \arg\max_{\pi} \overline{f}^{\,t}(\pi).$$

Initialization:

At $t = 1$,

$$n^t(\pi) = 0, \qquad \forall \pi \in [A],$$

and thus

$$\overline{f}^{\,1}(\pi) = \infty, \qquad \forall \pi \in [A].$$

In the first step, any arm may be selected since it is all infinity. Once a certain arm is selected, then the upper confidence bound $\overline{f}$ for that arm is finite. So at $t = 2$, any remaining arm maybe selected. In the first A steps, each arm will be selected once.

Proposition 1.3.3 (UCB regret)**.** *Assume $f^*(\pi) \in [0,1]$ and r^t is 1-sub-Gaussian. Then, with probability at least $1-\delta$,*

$$\mathrm{Reg} \lesssim \sqrt{AT \log(AT/\delta)}.$$

Comment: This gives a $T^{1/2}$ dependence, compared with the $T^{2/3}$ dependence of the ε-greedy bound. A smaller power of T is better because the regret grows more slowly. Thus, the UCB algorithm has a better regret bound than the ε-greedy algorithm.

Proof of Proposition 1.3.3. Using lemma 1.3.1 and our construction of the confidence bounds, the regret can be bounded by the sum of the confidence widths: with probability 1-δ,

$$f^*(\pi^*) - f^*(\pi^t) \le \overline{f}^t(\pi^t) - \underline{f}^t(\pi^t) \le 2\sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi^t)}}$$

On the other hand,

$$f^*(\pi^*) - f^*(\pi^t) \le 1.$$

Thus, with probability 1-δ,

$$f^*(\pi^*) - f^*(\pi^t) \le 2\sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi^t)}} \wedge 1$$

where $a \wedge b = \min\{a, b\}$.

Thus with probability 1-δ,

$$\begin{aligned}\mathrm{Reg} &= \sum_{t=1}^{T} \left[f^*(\pi^*) - f^*(\pi^t)\right] \\ &\lesssim \sum_{t=1}^{T} \left(2\sqrt{\frac{2\log(2T^2A/\delta)}{n^t(\pi^t)}} \wedge 1\right).\end{aligned}$$

here, the "$\wedge 1$" term appears because we can write off the regret for early rounds where $n^t(\pi^t) = 0$ as 1.

To be continued. □

Lemma 1.3.4 (Confidence width potential lemma)**.** *We have*

$$\sum_{t=1}^{T} \frac{1}{\sqrt{n^t(\pi^t)}} \wedge 1 \lesssim \sqrt{AT}.$$

Proof:

$$\sum_{t=1}^{T} \frac{1}{\sqrt{n^t(\pi^t)}} \wedge 1 = \sum_{\pi} \sum_{t=1}^{T} \frac{\mathbf{1}(\pi^t = \pi)}{\sqrt{n^t(\pi)}} \wedge 1.$$

Since

$$n^t(\pi^t) = \sum_{\pi} n^t(\pi)\mathbf{1}(\pi^t = \pi),$$

we can write

$$\frac{1}{\sqrt{n^t(\pi^t)}} = \sum_{\pi} \frac{\mathbf{1}(\pi^t = \pi)}{\sqrt{n^t(\pi)}}.$$

Therefore,

$$\sum_{t=1}^{T} \frac{1}{\sqrt{n^t(\pi^t)}} \wedge 1 = \sum_{\pi} \sum_{t=1}^{T} \frac{\mathbf{1}(\pi^t = \pi)}{\sqrt{n^t(\pi)}} \wedge 1.$$

Re-indexing according to the number of times arm π is selected,

$$= \sum_{\pi} \sum_{t=1}^{n^{T+1}(\pi)} \frac{1}{\sqrt{t-1}} \wedge 1.$$

Using

$$\sum_{t=1}^{n} \frac{1}{\sqrt{t-1}} \wedge 1 \leq 1 + 2\sqrt{n},$$

we obtain

$$\leq \sum_{\pi} \left(1 + 2\sqrt{n^{T+1}(\pi)}\right).$$

Thus,

$$= A + 2 \sum_{\pi} \sqrt{n^{T+1}(\pi)}.$$

By Jensen's inequality, since $F(x) = \sqrt{x}$ is concave,

$$\frac{1}{A} \sum_{\pi} \sqrt{n^{T+1}(\pi)} \leq \sqrt{\frac{1}{A} \sum_{\pi} n^{T+1}(\pi)}.$$

Hence,

$$A + 2 \sum_{\pi} \sqrt{n^{T+1}(\pi)} = A + 2A \left(\frac{1}{A} \sum_{\pi} \sqrt{n^{T+1}(\pi)}\right)$$

$$\leq A + 2A \sqrt{\frac{1}{A} \sum_{\pi} n^{T+1}(\pi)}.$$

Since

$$\sum_{\pi} n^{T+1}(\pi) = T,$$

we get

$$\leq A + 2A \sqrt{\frac{T}{A}} = A + 2\sqrt{AT}.$$

Finally, consider two cases:

1. If $A \leq \sqrt{AT}$ (equivalently, $A \leq T$),

then

$$A + 2\sqrt{AT} \leq 3\sqrt{AT}.$$

2. If $A > T$,

then trivially

$$\sum_{t=1}^{T} \frac{1}{\sqrt{n^t(\pi^t)}} \wedge 1 \leq T \leq \sqrt{AT}.$$

Therefore,

$$\boxed{\sum_{t=1}^{T} \frac{1}{\sqrt{n^t(\pi^t)}} \wedge 1 \leq 3\sqrt{AT}}$$

Chapter 2

Unconstrained optimization

2.1 Newton's method for unconstrained minimization (Section 9.5)

Now consider minimizing $f(x)$ without equality constraints. At a current point $x \in \mathrm{dom}(f)$, the Newton step is

$$\Delta x_{\mathrm{nt}} = -[\nabla^2 f(x)]^{-1} \nabla f(x). \tag{2.1}$$

Equivalently, it solves the linear system $\nabla^2 f(x) \Delta x_{\mathrm{nt}} = -\nabla f(x)$.

2.1.1 The Newton step is a descent direction

For a direction v, the directional derivative at x is

$$\left. \frac{d}{dt} f(x + tv) \right|_{t=0} = \langle \nabla f(x), v \rangle = \nabla f(x)^T v.$$

A negative value means that v is a descent direction. Substituting (2.1) yields

$$\left. \frac{d}{dt} f(x + t\Delta x_{\mathrm{nt}}) \right|_{t=0} = -\nabla f(x)^T [\nabla^2 f(x)]^{-1} \nabla f(x) \leq 0.$$

2.1.2 One-dimensional intuition

In one dimension, we seek a zero of $f'(x)$. Approximating f' by its tangent line at x_k gives

$$f'(x_k) + f''(x_k)(x_{k+1} - x_k) = 0, \qquad x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

The next iterate is the horizontal-axis intercept of that tangent line. Repeating the construction gives successive Newton iterates.
Geometrically, Newton's method uses a tangent line to approximate $f'(x)$ and find where $f'(x) = 0$. The tangent line has the form

$$g'(x) = ax + b.$$

This means that we are approximating $f(x)$ using a quadratic function of the form

$$g(x) = \frac{1}{2} a x^2 + bx + c.$$

The derivative of this quadratic is $g'(x) = ax + b$, so finding the zero of the tangent-line approximation of $f'(x)$ is equivalent to finding the minimum of the quadratic approximation of $f(x)$ when $a > 0$.

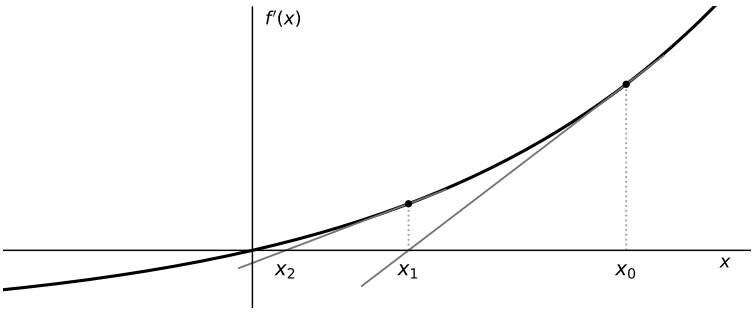


Figure 2.1: Illustration of the tangent-line construction applied to a general f'..

2.1.3 First interpretation: minimizing a quadratic approximation

The second-order Taylor approximation about x is

$$\widehat{f}(x+v) = f(x) + \nabla f(x)^T v + \frac{1}{2} v^T \nabla^2 f(x) v. \tag{2.2}$$

Treat the right-hand side as a function $g(v)$. Its gradient is

$$\nabla g(v) = \nabla f(x) + \nabla^2 f(x) v.$$

Because the Hessian of g is positive definite, the condition $\nabla g(v) = 0$ is necessary and sufficient for its unique minimum. Solving gives

$$v = -[\nabla^2 f(x)]^{-1} \nabla f(x) = \Delta x_{\mathrm{nt}}.$$

Thus the Newton step minimizes the local quadratic model.

2.1.4 Second interpretation: linearizing the optimality condition

For unconstrained convex minimization, the first-order optimality condition is $\nabla f(x+v) = 0$. The first order Taylor expansion gives

$$\nabla f(x+v) \approx \nabla f(x) + \nabla^2 f(x) v.$$

Setting this approximation equal to zero again gives $v = \Delta x_{\mathrm{nt}}$. Unlike the negative gradient direction, the Newton direction uses curvature information through the Hessian. If $\nabla^2 f(x) = I$, then $\Delta x_{\mathrm{nt}} = -\nabla f(x)$, simplifying to the standard gradient descent direction.

2.2 Newton decrement and stopping criterion

Definition 2.2.1. The Newton decrement at x is

$$\lambda(x) = \left(\nabla f(x)^T [\nabla^2 f(x)]^{-1} \nabla f(x) \right)^{1/2}. \tag{2.3}$$

It measures the improvement predicted by the quadratic approximation. Since (2.2) is minimized at the

Newton step,

$$\begin{aligned} f(x) - p^* = f(x) - \inf_y \widehat{f}(y) &= f(x) - \widehat{f}(x + \Delta x_{\text{nt}}) \\ &= -\nabla f(x)^T \Delta x_{\text{nt}} - \frac{1}{2} \Delta x_{\text{nt}}^T \nabla^2 f(x) \Delta x_{\text{nt}} \\ &= \nabla f(x)^T [\nabla^2 f(x)]^{-1} \nabla f(x) - \frac{1}{2} \nabla f(x)^T [\nabla^2 f(x)]^{-1} \nabla f(x) \\ &= \frac{1}{2} \lambda(x)^2. \end{aligned}$$

In simplifying the quadratic term, we use symmetry of the Hessian and its inverse.

2.2.1 Newton's method algorithm

Given a starting point $x \in \text{dom}(f)$ and tolerance $\epsilon > 0$, repeat:

1. Compute the Newton step and squared decrement:

$$\Delta x_{\text{nt}} = -[\nabla^2 f(x)]^{-1} \nabla f(x), \qquad \lambda(x)^2 = \nabla f(x)^T [\nabla^2 f(x)]^{-1} \nabla f(x).$$

2. Stop if $\lambda(x)^2/2 \leq \epsilon$.
3. Choose a step size t by backtracking line search.
4. Update $x \leftarrow x + t\Delta x_{\text{nt}}$.

2.3 Convergence discussion

Newton's method can converge much faster than gradient descent because it uses curvature information in addition to the gradient. Further convergence analysis will be discussed in the next lecture.

Chapter 3

Equality constrained optimization

3.1 Equality constrained minimization

3.1.1 Introduction

In this lecture, we begin the discussion of the methods for solving a convex optimization problem with equality constraints. We want to minimize $f(x)$,

$$\begin{aligned} &\min f(x) \\ &\quad \text{s.t. } Ax = b, \end{aligned}$$

where $A \in \mathbb{R}^{p \times n}$.

To compare this problem with the general framework discussed in the previous lectures, we can rewrite the problem as follows:

$$\begin{aligned} &\min f(x) \\ &\quad \text{s.t. } Ax - b = 0, \end{aligned}$$

where $A = \begin{bmatrix} a_1^T \\ a_2^T \\ \vdots \\ a_p^T \end{bmatrix}$ and a_i, $i \in \{1, 2, \ldots, p\}$ are column vectors.

Then

$$Ax - b = \begin{bmatrix} a_1^T x - b_1 \\ a_2^T x - b_2 \\ \vdots \\ a_p^T x - b_p \end{bmatrix} = 0. \tag{3.1}$$

From Eqn. (3.1) we can determine the equality constraint h_j:

$$h_j(x) = a_j^T x - b_j,$$

for $j \in \{1, 2, \ldots, p\}$

3.1.2 Equality constrained Convex Optimization

We are working with the convex optimization problem. To find a solution, make the following assumptions:

1. $f(x) : \mathbb{R}^n \to \mathbb{R}$ is convex and twice continuously differentiable
2. $A \in \mathbb{R}^{p \times n}$ has full row rank, i.e. it has linearly independent rows and $\text{rank}(A) = p < n$
3. Optimal solution $x^* \in \text{dom}\, f$ exists and $p^* = f(x^*)$ will produce the smallest value of $f(x)$.

To solve this problem, try using the KKT conditions.
The Lagrangian without the inequality constraint λ is:

$$\begin{aligned} L(x, \nu) &= f(x) + \sum_{j=1}^{p} \nu_j h_j(x) \\ &= f(x) + \sum_{j=1}^{p} \nu_j (a_j^T x - b_j) \\ &= f(x) + \nu^T (Ax - b) \\ &= f(x) + ((A^T \nu)^T x - \nu^T b), \end{aligned} \tag{3.2}$$

where $\nu \in \mathbb{R}^p$ is associated with the equality constraints.
The KKT conditions are as follows:

1. $\nabla_x L(x, \nu) = \nabla f(x) + A^T \nu = 0$, from (3.2)
2. NA since there are no inequality constraints,
3. $Ax = b$, from $h_j(x) = 0$, where $h_j = Ax - b$
4. NA since there are no inequality constraints,
5. NA since there are no inequality constraints.

Then (x^*, ν^*) satisfies

$$\begin{cases} \nabla f(x^*) + A^T \nu^* = 0 \\ Ax^* = b. \end{cases} \tag{3.3}$$

Since $x \in \mathbb{R}^n$ and $A \in \mathbb{R}^{p \times n}$,

$$\nabla f(x^*) + A^T \nu^* = 0$$

has n equations, and

$$Ax^* = b$$

has p equations.
Therefore, there are $n + p$ equations in the $n + p$ variables $x^*,\ \nu^*$.

3.1.3 Equality constrained convex quadratic minimization

Consider a simple example to build intuition.

10.1.1. Equality constrained convex quadratic minimization.

Let $f(x) = \frac{1}{2}x^T Px + q^T x + r$ be a quadratic function. We want to minimize $f(x)$,

$$\begin{aligned} &\min f(x) \\ &\text{s.t. } Ax = b, \end{aligned} \tag{3.4}$$

where $P \in \mathbb{S}_+^n$ and $A \in \mathbb{R}^{p\times n}$. Note that $\mathbb{S}_+^n$ are $n \times n$ PSD matrices with all eigenvalues ≥ 0.

Then

$$\nabla f(x) = Px + q$$

and

$$\nabla^2 f(x) = P,$$

hence, $\nabla^2 f(x)$ is PSD, which implies that $f(x)$ is convex.

Applying the derived KKT conditions in Eqn. (3.3) to this special case yields the following optimality conditions:

$$\begin{cases} Px^* + q + A^T\nu^* = 0 \\ Ax^* = b, \end{cases}$$

which implies

$$\begin{cases} Px^* + A^T\nu^* = -q \\ Ax^* + 0 \times \nu^* = b \end{cases} \tag{3.5}$$

The linear system of equations in Eqn. (3.5) can be rewritten in the matrix notation:

$$\begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x^* \\ \nu^* \end{bmatrix} = \begin{bmatrix} -q \\ b \end{bmatrix}, \tag{3.6}$$

where

$$\begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix}$$

is called the KKT matrix.

Solving this linear system of equations produces the desired solution to the equality constrained quadratic minimization problem.

There is a unique optimal primal-dual pair $(x^*,\ \nu^*)$ if the KKT matrix is non-singular. If the KKT matrix is singular and the system is solvable, any solution produces an optimal pair $(x^*,\ \nu^*)$. If the system is unsolvable, the quadratic optimization problem is unbounded below.

Lemma 3.1.1. *If (3.6) has no solution, then (3.4) is unbounded below.*

To prove this lemma, we need another lemma:

Lemma 3.1.2. *If a matrix $B \in \mathbb{R}^{d\times d}$ is symmetric and $y \notin \text{colspan}(B)$, then $\exists z\ Bz = 0$ and $\langle z,\ y\rangle > 0$.*

Proof. Recall: $B = \begin{bmatrix} b_1 & b_2 & \dots & b_d \end{bmatrix}$, where b_j for $j \in \{1, 2, \dots, d\}$ are column vectors. Then

$$\begin{aligned} \text{colspan}(B) &= \{\sum_{j=1}^{d} \lambda_j b_j \mid \lambda_j \in \mathbb{R}\} \\ &= \{B\lambda \mid \lambda \in \mathbb{R}^d\}. \end{aligned} \tag{3.7}$$

From Eqn. (3.7),

$$y \notin \text{colspan}(B) \Leftrightarrow B\lambda = y \text{ has no solution.}$$

Since $y \notin \text{colspan}(B)$, $\exists\, y_1, y_2$ $y = y_1 + y_2$, where $y_1 \in \text{colspan}(B)$ and $y_2 \perp \text{colspan}(B)$.

In other words, we can decompose the vector y into components parallel and perpendicular to colspan(B) as in Fig. (3.1).

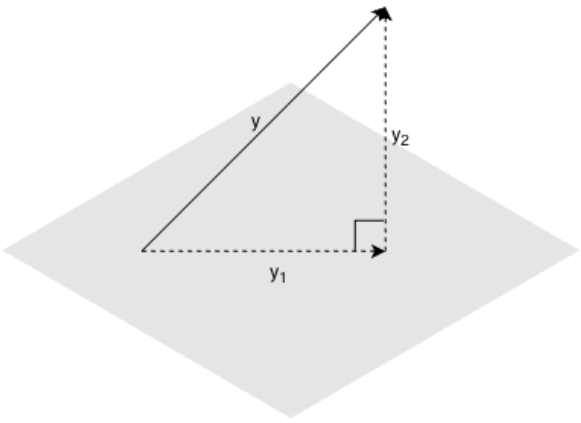


Figure 3.1: Vector y is decomposed into components y_1 (parallel to the column span of B) and y_2 (perpendicular to the column span of B).

Assume $y_2 = 0$. Then there is no component of y perpendicular to colspan(B), which means $y = y_1 \in$ colspan(B). It contradicts the assumption that $y \notin$ colspan(B). Therefore, $y_2 \neq 0$.

Since $y_2 \perp$ colspan(B), $\langle y_2, b_i \rangle = 0$ for $i \in \{1, 2, \dots, d\}$.

The equation above can be rewritten as

$$\begin{bmatrix} b_1^T y_2 \\ b_2^T y_2 \\ \vdots \\ b_d^T y_2 \end{bmatrix} = 0 \Leftrightarrow B^T y_2 = 0 \Leftrightarrow B y_2 = 0$$

since B is symmetric.

Now, check that $\langle y_2, y \rangle > 0$.

$$\begin{aligned} \langle y_2, y \rangle = \langle y_2, y_1 + y_2 \rangle &= \langle y_2, y_1 \rangle + \langle y_2, y_2 \rangle \\ &= 0 + \langle y_2, y_2 \rangle \\ &= \|y_2\|^2 > 0, \end{aligned}$$

since $y_2 \neq 0$ and $y_2 \perp y_1 \implies \langle y_2, y_1 \rangle = 0$.

□

Using Lemma 4.2, we can prove Lemma 4.1.

Proof. If (3.6) has no solution, then

$$\begin{bmatrix} -q \\ b \end{bmatrix} \notin \text{colspan} \begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix}.$$

By Lemma 4.2, $\exists z = \begin{bmatrix} v \\ w \end{bmatrix}$ such that

$$\begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

and

$$\langle \begin{bmatrix} v \\ w \end{bmatrix}, \begin{bmatrix} -q \\ b \end{bmatrix} \rangle > 0 \Leftrightarrow -q^T v + b^T w > 0 \tag{3.8}$$

Let $\hat{x}$ be feasible point of (3.4), i.e. $\hat{x}$ satisfies the constraint $A\hat{x} = b$.
Then $\hat{x} + tv$ is also feasible for all t, where v is a particular direction and t is the step size.
Indeed,

$$\begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} v \\ w \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \implies \begin{cases} Pv + A^T w = 0 \\ Av = 0 \end{cases} \implies \begin{cases} Pv = -A^T w \\ Av = 0 \end{cases} \tag{3.9}$$

and

$$A(\hat{x} + tv) = A(\hat{x}) + t(Av) = b + t \times 0 = b,$$

which implies that $\hat{x} + tv$ is feasible.
Then

$$\begin{aligned} f(\hat{x} + tv) &= f(\hat{x}) + t(v^T P\hat{x} + q^T v) + \frac{1}{2}t^2 v^T P v \\ &= f(\hat{x}) + t(\hat{x}^T P v + q^T v) + \frac{1}{2}t^2 v^T P v \\ &= f(\hat{x}) + t(\hat{x}^T(-A^T w) + q^T v) + \frac{1}{2}t^2 v^T(-A^T w) && \text{from (3.9)} \\ &= f(\hat{x}) + t(-\hat{x}^T A^T w + q^T v) - \frac{1}{2}t^2 w^T A v \\ &= f(\hat{x}) + t(-b^T w + q^T v) - 0 && \text{using } A\hat{x} = b \text{ and } Av = 0 \end{aligned}$$

Since $(-q^T v + b^T w) > 0$ by Eqn. (3.8), we have $(-b^T w + q^T v) < 0$. Then

$$\lim_{t \to \infty} f(\hat{x} + tv) = -\infty.$$

Therefore, the quadratic optimization problem is unbounded below. □

3.2 Eliminating equality constraints

3.2.1 Review: equality-constrained minimization

Consider

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & Ax = b, \end{array} \tag{3.10}$$

where f is convex and twice continuously differentiable, $A \in \mathbb{R}^{p\times n}$, and $\operatorname{rank}(A) = p$. Thus the equality constraints have no redundant rows. At a primal optimum x^*, the KKT conditions are

$$\nabla f(x^*) + A^T\nu^* = 0, \qquad Ax^* = b, \tag{3.11}$$

where $\nu^* \in \mathbb{R}^p$ is dual optimal.

3.2.2 Review: Quadratic objective

For the quadratic objective

$$f(x) = \frac{1}{2}x^T Px + q^T x + r,$$

the gradient is $Px + q$. Consequently, (3.11) becomes the linear system

$$\begin{bmatrix} P & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} x^* \\ \nu^* \end{bmatrix} = \begin{bmatrix} -q \\ b \end{bmatrix}. \tag{3.12}$$

The block matrix has size $(n+p)\times(n+p)$. If it is invertible, the system has a unique solution. If the feasible convex quadratic problem has no solution to (3.12), its objective is unbounded below.

3.2.3 Parameterizing the feasible set (Section 10.1.2)

Choose any particular solution $\widehat{x}$ of $Ax = b$, and let $F \in \mathbb{R}^{n\times(n-p)}$ have columns forming a basis of the null space of A. In particular,

$$AF = 0, \qquad \operatorname{range}(F) = \operatorname{null}(A).$$

Every point can be written as

$$\{x : Ax = b\} = \{Fz + \widehat{x} : z \in \mathbb{R}^{n-p}\}. \tag{3.13}$$

To verify, take $x = Fz + \widehat{x}$. Then

$$Ax = A(Fz + \widehat{x}) = AFz + A\widehat{x} = 0 + b = b.$$

Define $\widetilde{f}(z) = f(Fz + \widehat{x})$. Problem (3.10) is equivalent to the unconstrained problem

$$\underset{z\in\mathbb{R}^{n-p}}{\text{minimize}} \quad \widetilde{f}(z).$$

After finding a minimizer z^*, we recover a minimizer of the original problem as $x^* = Fz^* + \widehat{x}$.

3.2.4 Optimal allocation with a resource constraint (Example 10.1)

Consider the separable objective and resource constraint

$$\begin{array}{ll} \text{minimize} & \displaystyle\sum_{i=1}^n f_i(x_i) \\ \text{subject to} & \displaystyle\sum_{i=1}^n x_i = b. \end{array}$$

Here $A = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}$ has rank one. Eliminating the last variable gives

$$x_n = b - \sum_{i=1}^{n-1} x_i,$$

and hence the unconstrained function in $n-1$ variables is

$$\sum_{i=1}^{n-1} f_i(x_i) + f_n\left(b - \sum_{i=1}^{n-1} x_i\right).$$

Now the constraint is removed.
This substitution can be written in the format of (3.13). Let e_n be the last canonical basis vector in $\mathbb{R}^n$, and choose

$$\widehat{x} = be_n = \begin{bmatrix} 0 \\ \vdots \\ 0 \\ b \end{bmatrix}, \qquad F = \begin{bmatrix} I_{n-1} \\ -\mathbf{1}^T \end{bmatrix} \in \mathbb{R}^{n \times (n-1)},$$

where $\mathbf{1} \in \mathbb{R}^{n-1}$ is the vector of ones. We have $A\widehat{x} = b$, $AF = 0$, and the columns of F are linearly independent, so they form a basis of $\text{null}(A)$. Therefore

$$Fz + \widehat{x} = \begin{bmatrix} z \\ b - \sum_{i=1}^{n-1} z_i \end{bmatrix}, \qquad z \in \mathbb{R}^{n-1},$$

which recovers precisely the elimination formula above.

Chapter 4

PCA

4.1 Introduction

We have a high dim data set. It distributes around a low dim set.
Dimension Reduction: Reorganize the data in a low dim space. We want to preserve some properties of the data set in the low dim representation.
Principal Component Analysis (PCA).

1. Algorithm.
2. In what sense it reorganizes the data.
3. Efficient algorithm through SVD.

PCA is a linear dimension reduction method.
Data set:

$$\{X_1, \dots, X_n\} \subset \mathbb{R}^p, \qquad p \text{ is large.}$$

$$X_i = \begin{bmatrix} \vdots \\ \vdots \\ \vdots \end{bmatrix}, \qquad p \text{ vector.}$$

$$\mu_n = \frac{1}{n} \sum_{i=1}^{n} X_i \in \mathbb{R}^p.$$

4.2 Sample Covariance Matrix

$$C_n = \sum_{i=1}^{n} \underbrace{(X_i - \mu_n)}_{\mathbb{R}^{p\times 1}} \underbrace{(X_i - \mu_n)^T}_{\mathbb{R}^{1\times p}} \in \mathbb{R}^{p\times p}. \tag{*}$$

$$\Sigma_n = \frac{1}{n-1} C_n \qquad \text{Sample covariance matrix.}$$

Properties of C_n.

1. C_n is symmetric.

2.

$$C_n = U\Lambda U^T, \qquad U \in O(p).$$

U is an orthogonal $p \times p$ matrix, i.e.

$$UU^T = U^T U = I_{p\times p}.$$

$$\Lambda = \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_p \end{bmatrix}, \qquad U = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_p \\ | & & | \end{bmatrix}.$$

$$\lambda_1 \geq \cdots \geq \lambda_p \geq 0.$$

$\{u_1, \ldots, u_p\}$ are orthonormal eigenvectors, i.e. C_n is positive semi-definite. This follows from the structure of C_n in $(*)$ (e.g. eigenvalues ≥ 0).

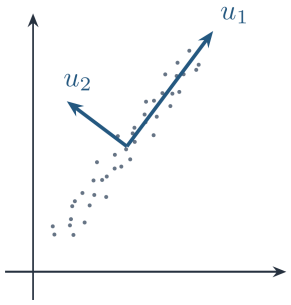
$$(\lambda_i, u_i)_{i=1}^p \text{ are the orthonormal eigenpairs of } C_n.$$

4.2.1 Understand Σ_n (or C_n)

$$u_i: \text{ direction of the } i\text{th largest variance of the data } \{X_1, \ldots, X_n\}.$$

$$\underbrace{\frac{\lambda_i}{n-1}}_{\text{eigenvalue of } \Sigma_n} : \quad i\text{th largest variance.}$$

$$p = 2, \qquad \{X_1, \ldots, X_n\} \subset \mathbb{R}^2$$



4.3 PCA Algorithm

Input: $\{X_1, \ldots, X_n\} \subset \mathbb{R}^p$.
of principal components: k.
Step 1. Construct C_n using $\{X_1, \ldots, X_n\}$.
Step 2. Find the eigendecomp of C_n:

$$C_n = U\Lambda U^T.$$

$$U = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_p \\ | & & | \end{bmatrix}, \qquad \Lambda = \begin{bmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_p \end{bmatrix}, \qquad \lambda_1 \geq \cdots \geq \lambda_p \geq 0.$$

Step 3. Project $X_i - \mu_n$ onto the first k eigenvectors.

$$\widetilde{U} = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_k \\ | & & | \end{bmatrix}.$$

Projection:

$$\widetilde{U}^T(X_i - \mu_n) = \widetilde{X}_i \in \mathbb{R}^k.$$

$$X_i \xrightarrow{\text{PCA}} \widetilde{X}_i = \begin{bmatrix} u_1^T(X_i - \mu_n) \\ \vdots \\ u_k^T(X_i - \mu_k) \end{bmatrix} \in \mathbb{R}^k.$$

4.4 In What Sense PCA Reorganizes the Data

PCA reduce the dimension of the data from $\mathbb{R}^p$ to $\mathbb{R}^k$.

$$\sum_{i=1}^{n} \widetilde{X}_i = \widetilde{U}^T \left(\underbrace{\sum_{i=1}^{n} (X_i - \mu_n)}_{=0} \right) = 0.$$

PCA reorganize the data in the following sense.

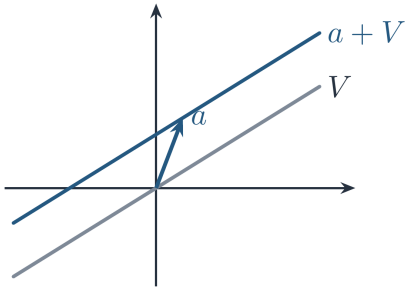
1. **Projection with maximum variability.** Project $\{X_1 - \mu_n, \ldots, X_n - \mu_n\}$ onto the principal components corresponding to the directions with k largest variances of the data.
2. It finds a k-dim affine subspace of $\mathbb{R}^p$ that best fits the data in the least square sense.

4.4.1 Recall Affine Space

V is a vector subspace of $\mathbb{R}^p$, $a \in \mathbb{R}^p$.

$$\text{affine subspace} \qquad a + V = \{a + v \mid v \in V\}.$$

$$\dim(a + V) = \dim(V).$$



$$\dim(V) = k, \qquad V \text{ is a subspace of } \mathbb{R}^p.$$

$\{\widetilde{u}_1, \ldots, \widetilde{u}_k\}$ form an orthonormal basis of V.

$$\beta_i \in \mathbb{R}^k, \qquad i = 1, \ldots, n.$$

$$\beta_i = \begin{bmatrix} \beta_{i,1} \\ \vdots \\ \beta_{i,k} \end{bmatrix} \in \mathbb{R}^k, \qquad \widetilde{U} = \begin{bmatrix} | & & | \\ \widetilde{u}_1 & \cdots & \widetilde{u}_k \\ | & & | \end{bmatrix}.$$

$$y_i = a + \widetilde{U}\beta_i = a + \sum_{j=1}^{k} \beta_{i,j}\widetilde{u}_j \in a + V.$$

4.4.2 Least Square Metric

Given $\{X_1, \ldots, X_n\} \subset \mathbb{R}^p$ and an affine subspace $a + V$ of $\mathbb{R}^p$,

$$\{y_1, \ldots, y_n\} \subset a + V.$$

$$\widetilde{U} = \begin{bmatrix} | & & | \\ \widetilde{u}_1 & \cdots & \widetilde{u}_k \\ | & & | \end{bmatrix}.$$

$\{\widetilde{u}_i\}$ are orthonormal basis of V. We can find $\beta_1, \ldots, \beta_n \in \mathbb{R}^k$ s.t.

$$y_i = a + \widetilde{U}\beta_i.$$

$$I\Big(\underbrace{a, \widetilde{U}}_{\substack{\text{determines the affine} \\ \text{subspace } a+V}}, \underbrace{\beta_1, \ldots, \beta_n}_{\text{determines } y_i \in a+V} \Big) = \sum_{i=1}^{n} \|X_i - y_i\|_{\mathbb{R}^p}^2$$

$$= \boxed{\sum_{i=1}^{n} \|X_i - a - \widetilde{U}\beta_i\|_{\mathbb{R}^p}^2}.$$

4.4.3 PCA Minimizes the Least Square Metric

Theorem 4.4.1. *Given $\{X_1, \ldots, X_n\} \subset \mathbb{R}^p$, let*

$$a = \mu_n = \frac{1}{n}\sum_{i=1}^{n} X_i \in \mathbb{R}^p, \qquad \widetilde{U} = \begin{bmatrix} | & & | \\ u_1 & \cdots & u_k \\ | & & | \end{bmatrix} \in \mathbb{R}^{p \times k}.$$

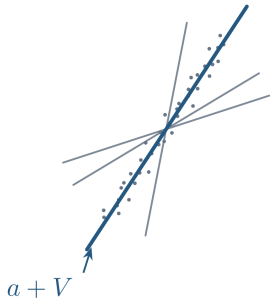
$u_1, \ldots, u_k$ are the orthonormal eigenvectors of C_n corresponding to the decreasing order of the eigenvalues.

$$\beta_i = \widetilde{X}_i \qquad \textit{(output of PCA).}$$

Then $I(a, \widetilde{U}, \beta_1, \ldots, \beta_n)$ is minimized.

I.e. PCA finds $y_1, \ldots, y_n$ in the affine space $\mu_n + V$ with coordinates $\beta_1, \ldots, \beta_n$ and V generated by $u_1, \ldots, u_k$ so that

$$\sum_{i=1}^{n} \|X_i - y_i\|^2 \text{ is minimized.}$$



4.5 Complexity of the PCA Algorithm

Step 1.

$$\mu_n = \frac{1}{n}\sum_{i=1}^{n} X_i, \qquad C_n = \sum_{i=1}^{n}(X_i - \mu_n)(X_i - \mu_n)^T.$$

Recall the Big O notation. Complexity (# of operations, e.g. addition and multiplication).

Complexity in step 1:

$$\underbrace{n}_{\mu_n} + \underbrace{p \cdot p \cdot n}_{C_n} \leq 2p^2 n = O(p^2 n).$$

Step 2. Eigendecomposition. Complexity $O(p^3)$.

Step 3. Projection. Complexity is dominated by the complexity in the first 2 steps.

Total complexity is

$$O(p^2 n + p^3).$$

4.6 Singular Value Decomposition (SVD)

$$M \in \mathbb{R}^{p \times n}.$$

$$\text{SVD:} \qquad M = USV^T.$$

$$\begin{aligned} U: &\quad p \times p \text{ orthogonal matrix,} \\ V: &\quad n \times n \text{ orthogonal matrix,} \\ &S \in \mathbb{R}^{p \times n}. \end{aligned}$$

$$S = \underbrace{\begin{bmatrix} \sigma_1 & & 0 \\ & \ddots & \\ 0 & & \sigma_{\min(p,n)} \\ & 0 & \end{bmatrix}}_{p \geq n} \quad \text{or} \quad \underbrace{\begin{bmatrix} \sigma_1 & & 0 & \\ & \ddots & & 0 \\ 0 & & \sigma_{\min(p,n)} & \end{bmatrix}}_{p \leq n}.$$

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_{\min(p,n)} \geq 0.$$

They are singular values of M. Column vectors of U (or V) are the left (right) singular vectors.

$$Y = \begin{bmatrix} | & & | \\ X_1 - \mu_n & \cdots & X_n - \mu_n \\ | & & | \end{bmatrix} \in \mathbb{R}^{p \times n}.$$

Observation: $\boxed{C_n = YY^T}$.

SVD of Y:

$$Y = \widetilde{U}\Sigma\widetilde{V}^T.$$

$$\begin{aligned} C_n &= \widetilde{U}\Sigma\underbrace{\widetilde{V}^T\widetilde{V}}_{I_{n\times n}}\Sigma^T U^T \\ &= \widetilde{U}\Sigma\Sigma^T\widetilde{U}^T \\ &= U\Lambda U^T \qquad \text{eigendecomposition.} \end{aligned}$$

$$\implies \underbrace{\lambda_i}_{\text{eigenvalues of } C_n} = \sigma_i^2 \geq 0, \qquad \Lambda = \Sigma\Sigma^T, \qquad U = \widetilde{U}.$$

First k singular vectors of Y can be calculated in $O(pnk)$.

$$pnk \leq p^2 n + p^3.$$

Use SVD of Y to replace the first two steps of PCA.

Bibliography

[1] S. Boyd, L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004.

[2] D. J. Foster and A. Rakhlin, *Foundations of reinforcement learning and interactive decision making*, arXiv preprint arXiv:2312.16730, 2023. Sections 2.2 and 2.3.